

The hardware for arithmetic operations mainly consists of:

1. Binary Adder
2. Complement Circuit
3. Multiplexers
4. Registers
5. Control Logic

Binary Adder

Definition

A **Binary Adder** is a digital circuit used to perform the **addition of binary numbers**. It is the core component of the arithmetic unit.

Types of Binary Adders

1. Half Adder
2. Full Adder
3. Parallel (Ripple Carry) Adder

1. Half Adder

Definition

A **Half Adder** is a combinational circuit used to **add two single binary digits** and produce **two outputs: Sum and Carry**.

It is called a **half adder** because it **cannot include a carry input from a previous stage**.

Inputs and Outputs

Inputs:

- A
- B

Outputs:

- Sum (S)

- Carry (C)

Truth Table

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expressions

Sum:

$$S = A \oplus B$$

Carry:

$$C = A \cdot B$$

2. Full Adder

Definition

A **Full Adder** is a combinational circuit that **adds three binary digits**:

- Two input bits
- One carry input from the previous stage

It produces:

- **Sum**
- **Carry output**

Inputs and Outputs

Inputs:

- A
- B

- Carry In (C_{in})

Outputs:

- Sum (S)
- Carry Out (C_{out})

Truth Table

A B C_{in} Sum C_{out}

0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Expressions

Sum:

$$S = A \oplus B \oplus C_{in}$$

Carry:

$$C_{out} = AB + BC_{in} + AC_{in}$$

Role in Arithmetic Unit

- Performs **addition of two binary numbers**
- Used as the base circuit for **adder-subtractor design**
- Several full adders connected together form an **n-bit parallel adder**

3. Parallel Adder (Ripple Carry Adder)

Definition

A **Parallel Adder** (also called **Ripple Carry Adder**) is a circuit used to **add multi-bit binary numbers**.

It consists of **multiple full adders connected in series**.

The **carry output of one stage becomes the carry input of the next stage**.

Example: 4-bit Ripple Carry Adder

Inputs:

- A3 A2 A1 A0
- B3 B2 B1 B0

2. Complement Circuit

Definition

A **Complement Circuit** is used to generate the **1's complement** or **2's complement** of a binary number.

This is mainly required for performing **subtraction operations**.

1's Complement

Inverts each bit of the number.

Example:

Binary number: 101010

1's complement: 010101

2's Complement

$$2's\ complement = 1's\ complement + 1$$

Example:

Number = 0101

1's complement = 1010

Add 1 → 1011

3. Multiplexers

Definition

A **Multiplexer (MUX)** is a combinational circuit that selects **one input from multiple inputs** and forwards it to the output based on a **selection signal**.

Example: 4-to-1 Multiplexer

Inputs:

- I0
- I1
- I2
- I3

Selection lines:

- S0
- S1

Output:

- Y

Operation Table

S1 S0 Output

0 0 I0

0 1 I1

1 0 I2

1 1 I3

4. Registers

Definition

A **Register** is a group of **flip-flops used to store binary data temporarily**.

Registers are used to store:

- Operands

- Intermediate results
- Final results

Types of Registers Used

1. **Accumulator Register (A Register)**
2. **B Register**
3. **Result Register**

Example

A Register → stores first operand

B Register → stores second operand

Result Register → stores output

Characteristics

- Stores **n-bit binary numbers**
- Operates synchronously with **clock signals**
- Allows **data transfer between components**

Role in Arithmetic Unit

Registers:

- Hold the input operands
- Store intermediate results
- Transfer data to and from the arithmetic unit

5. Control Logic

Definition

Control Logic is responsible for **controlling and coordinating all arithmetic operations** in the arithmetic unit.

It generates control signals that determine which operation should be performed.

Functions of Control Logic

1. Select arithmetic operation

2. Control multiplexers
3. Enable registers
4. Control complement circuits
5. Manage carry input

Example Control Signals

Control Signal	Operation
M = 0	Addition
M = 1	Subtraction

Working

The control unit sends signals to:

- Complement circuit
- Multiplexer
- Adder
- Registers

to perform the desired arithmetic operation.